

VOXA

An AI Pipeline for Accessible Speech-to-Text
Transcription · Simplification · Summarization

Project Documentation

Generative AI Course — Multi-Agent System Design

Digital Egypt Pioneers Initiative (DEPI)

Generative AI Track

Table of Contents

1. Project Overview	4
1.1 The Problem It Solves	4
1.2 Why a Multi-Agent Design	4
1.3 What the System Produces	4
2. System Architecture	5
2.1 The Orchestrator Pattern	5
2.2 Why Simplification and Summarization Both Read the Transcript	6
2.3 Configuration-Driven Design	6
3. Project File Structure	7
3.1 Folder Responsibilities	7
4. Detailed File-by-File Explanation	8
4.1 config.py	8
4.2 agents/transcription_agent.py — Agent 1	9
4.3 agents/simplification_agent.py — Agent 2	10
4.4 agents/summarization_agent.py — Agent 3	12
4.5 core/orchestrator.py	14
4.6 run_pipeline.py	15
4.7 ui/streamlit_app.py	15
5. Libraries & Technologies Used	17
5.1 Core AI Models	17
5.2 Audio Processing	17
5.3 Text Processing & Utilities	18

5.4 Web Interface	18
6. Setup & Running the Project	19
6.1 Environment Setup	19
6.2 Adding Trained Models	19
6.3 Running the Web Interface	19
6.4 Running from the Command Line	19
6.5 Testing Individual Agents	19
7. Team Roles & Responsibilities	20
7.1 Integration Contract	20
8. Future Improvements	21

1. Project Overview

Voxa is an end-to-end AI pipeline that converts spoken audio into clear, accessible text. It was built to help deaf and hard-of-hearing individuals access spoken content — lectures, meetings, videos, or everyday recordings — by automatically transcribing the audio, simplifying the language, and producing a short summary of the key points.

1.1 The Problem It Solves

Spoken audio is inaccessible to anyone who cannot hear it. Existing captioning tools often produce text that is accurate but still complex — long sentences, advanced vocabulary, and no summary of what actually matters. Voxa addresses three gaps at once:

- **Accuracy:** convert speech to text reliably using a trained transcription model.
- **Accessibility:** rewrite that text in plain, simple language that is easier to read and understand.
- **Efficiency:** produce a short summary so someone can quickly grasp the key points without reading everything.

1.2 Why a Multi-Agent Design

Instead of writing one large script that does everything, Voxa is split into three independent **agents**, each responsible for exactly one task. This mirrors how a real team works: one person owns transcription, another owns simplification, another owns summarization — and a coordinator (the **Orchestrator**) connects their work into a single pipeline. This design has concrete benefits for a course project built by a team:

- **Parallel teamwork** — each team member builds and tests their agent independently.
- **Isolated debugging** — a bug in one agent doesn't require understanding the other two.
- **Swappable components** — any agent's underlying model can be replaced without changing the others.
- **Clear grading:** each file has one obvious job, easy to explain in a discussion.

1.3 What the System Produces

Given one audio file, Voxa produces three outputs, plus supporting metadata:

Output	Produced By	Description
Transcript	TranscriptionAgent	The literal spoken words, converted to text by Whisper.
Simplified Text	SimplificationAgent	The transcript rewritten in short sentences and plain vocabulary.
Summary	SummarizationAgent	A condensed version containing only the key points.

2. System Architecture

Voxa follows a simple, linear pipeline: one audio file goes in, and three text outputs come out. The diagram below shows how the pieces connect.

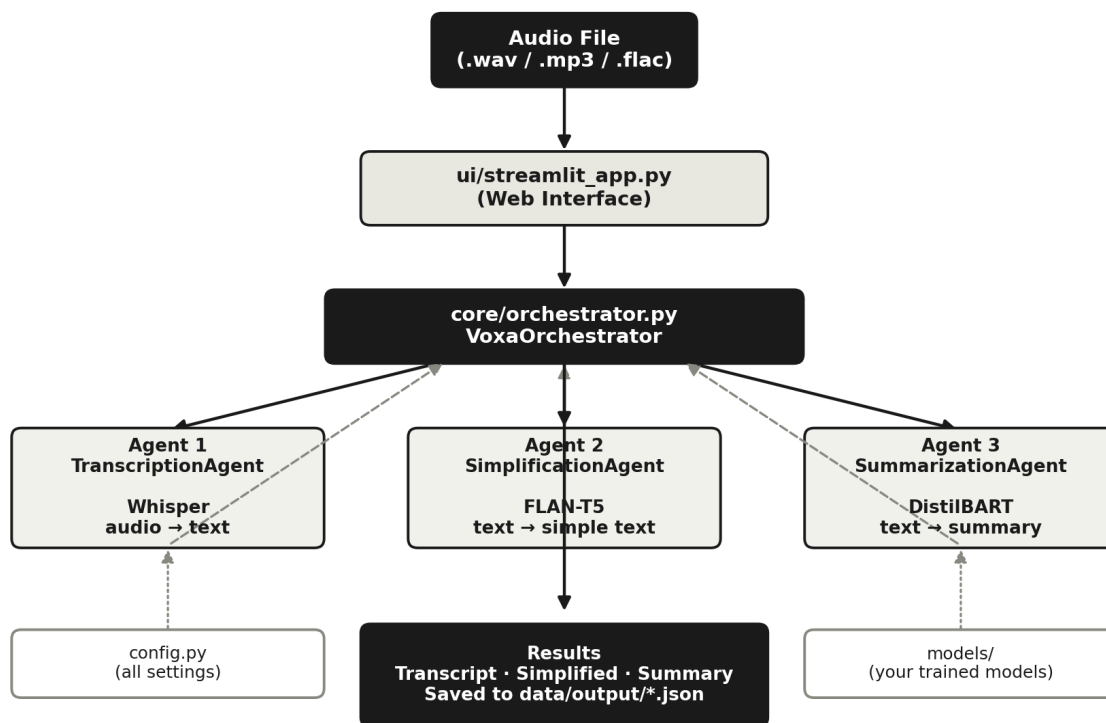


Figure 1 — Voxa system architecture: audio flows through the Streamlit UI, into the Orchestrator, which runs the three agents and collects their results.

2.1 The Orchestrator Pattern

The **Orchestrator** (`core/orchestrator.py`) is the only piece of code that knows about all three agents. It does not do any AI processing itself — its only job is to call each agent in the right order and pass data between them. This is a common software design pattern: separating *coordination logic* (which agent runs when) from *task logic* (what each agent actually does).

The sequence the Orchestrator follows is always the same:

- **Step 1:** Pass the audio file path to `TranscriptionAgent`. Receive back the transcript.
- **Step 2:** Pass that transcript to `SimplificationAgent`. Receive back the simplified text.
- **Step 3:** Pass the same transcript to `SummarizationAgent`. Receive back the summary.
- **Step 4:** Combine all three results into one dictionary, save it as JSON, and return it to whoever called `.run()` — either the CLI script or the Streamlit UI.

2.2 Why Simplification and Summarization Both Read the Transcript

Notice in the diagram that both Agent 2 and Agent 3 receive the *same* transcript directly from the Orchestrator — simplification does not feed into summarization, or vice versa. This is intentional: they are two independent transformations of the original text, not a chain. Keeping them independent means either one can fail, be skipped, or be replaced without affecting the other.

2.3 Configuration-Driven Design

Every adjustable setting in the project — which Whisper size to use, whether to use a pretrained or custom model, chunk sizes, output paths — lives in a single file: `config.py`. No agent file contains hardcoded settings. This means the entire system's behavior can be changed by editing one file, without touching any of the agent logic.

3. Project File Structure

The project follows a modular folder layout — each folder has one clear responsibility, which makes the codebase easy to navigate and easy to divide among a team.

```
voxa_team/
|
|-- config.py           All settings (model paths, chunk sizes, etc.)
|-- requirements.txt    Exact pinned library versions
|-- run_pipeline.py     Command-line runner (no UI)
|
|-- models/            Trained models are placed here
|   |-- README.md       Instructions for adding trained models
|   |-- whisper_finetuned/ (optional) fine-tuned whisper checkpoint
|   |-- simplification_model/ Trained FLAN-T5 model files
|   |-- summarization_model/ Trained DistilBART model files
|
|-- agents/            The 3 AI agents (one file each)
|   |-- transcription_agent.py Agent 1: Audio -> Text
|   |-- simplification_agent.py Agent 2: Text -> Simple Text
|   |-- summarization_agent.py Agent 3: Text -> Summary
|
|-- core/
|   |-- orchestrator.py  Runs the 3 agents in the correct order
|
|-- ui/
|   |-- streamlit_app.py The web interface
|
|-- data/
|   |-- input/           Test audio files go here
|   |-- output/          Results are saved here automatically
```

3.1 Folder Responsibilities

Folder / File	Responsibility
config.py	Central settings file. Every other file reads its configuration from here.
models/	Storage location for trained model files. Each agent checks this folder on startup.
agents/	One Python file per AI agent. Each file is self-contained and independently testable.
core/orchestrator.py	Coordinates the 3 agents. Contains no AI logic of its own.
ui/streamlit_app.py	Web interface. Imports the Orchestrator and displays its results.
run_pipeline.py	Command-line alternative to the web interface, useful for testing and automation.
data/input/ , data/output/	Working folders for test audio files and generated results.

4. Detailed File-by-File Explanation

This section explains every file in the project: its purpose, how its code works step by step, and which libraries it depends on.

4.1 config.py

Purpose: the single source of truth for every adjustable setting in the project. No agent file contains a hardcoded model name, chunk size, or file path — they all import values from here.

```
TRANSCRIPTION_CONFIG = {
    "model_size": "base",          # tiny / base / small / medium / large
    "use_pretrained": True,        # False = use YOUR fine-tuned model
    "model_path": TRANSCRIPTION_MODEL_PATH,
    "language": "en",
    "sample_rate": 16000,         # Whisper requires 16 kHz audio
}

SIMPLIFICATION_CONFIG = {
    "model_name": "google/flan-t5-base",
    "use_pretrained": True,        # False = use YOUR trained model
    "model_path": SIMPLIFICATION_MODEL_PATH,
    "chunk_size": 80,             # words per chunk sent to the model
    "max_length": 130,
    "num_beams": 4,               # beam search width
}
```

How it works: each configuration block is a plain Python dictionary. Every agent reads its section on startup (for example `config.TRANSCRIPTION_CONFIG`) and uses those values instead of hardcoded numbers. To change the Whisper model size for the entire project, only one line here needs to change — nothing in agents/ needs to be touched.

KEY SETTING: Setting `use_pretrained: False` tells an agent to load your own trained model from the corresponding folder in `models/` instead of the public Hugging Face model.

4.2 agents/transcription_agent.py — Agent 1

Purpose: converts an audio file into text. This is the first stage of the pipeline — every other agent depends on its output.

Libraries used: `whisper` (OpenAI's speech recognition model), `librosa` (audio loading and resampling), `json` (saving results).

How the model loads

On startup, the agent checks `config.TRANSCRIPTION_CONFIG`. If `use_pretrained` is `True`, it loads the public Whisper model at the configured size:

```
self.model = whisper.load_model(self.config["model_size"])
```

If `False`, it looks in `models/whisper_finetuned/` for a `.pt` checkpoint file (a fine-tuned Whisper model saved in PyTorch's native format) and loads that instead. If no checkpoint is found, it safely falls back to the pretrained model and prints a warning — the pipeline never crashes due to a missing custom model.

The `transcribe()` method — step by step

- **Step 1:** `_load_audio()` uses `librosa` to read the file and resample it to 16,000 Hz, the sample rate Whisper requires regardless of the original recording's rate.
- **Step 2:** The audio array is passed to `self.model.transcribe()`, which returns the full text plus a list of timed segments.
- **Step 3:** `_calculate_confidence()` filters out segments Whisper marked as likely silence (`no_speech_prob > 0.6`) and averages the remaining segments' log-probabilities into one confidence score between 0 and 1.
- **Step 4:** The method returns a dictionary containing the transcript, segments, detected language, confidence score, and word count.

```
result = self.model.transcribe(
    audio,
    language=self.config["language"],
    fp16=False,  # fp32 for CPU; set True if running on GPU
)

output = {
    "transcript": result["text"].strip(),
    "segments": result.get("segments", []),
    "language": result.get("language", "unknown"),
    "confidence": confidence,
    "word_count": len(result["text"].split()),
}
```

PERFORMANCE NOTE: `fp16=False` forces 32-bit floating point math, which is required for CPU-only machines. If your machine has an NVIDIA GPU, changing this to `True` roughly doubles transcription speed.

4.3 agents/simplification_agent.py — Agent 2

Purpose: rewrites the transcript in plain, easy-to-read language — shorter sentences, simpler vocabulary — while preserving the original meaning.

Libraries used: transformers (Hugging Face's model library, providing pipeline, AutoTokenizer, and AutoModelForSeq2SeqLM).

How the model loads

Like the transcription agent, this checks `use_pretrained` in config. If True, it builds a Hugging Face pipeline directly from the public model name (`google/flan-t5-base`). If False, it loads your own trained model folder using two components together:

```
self.tokenizer = AutoTokenizer.from_pretrained(model_path)
model_raw = AutoModelForSeq2SeqLM.from_pretrained(model_path)
self.model = pipeline(
    "text2text-generation",
    model=model_raw,
    tokenizer=self.tokenizer,
    framework="pt"
)
```

The **tokenizer** converts text into numeric tokens the model understands (and back again), while `AutoModelForSeq2SeqLM` loads the actual trained weights. Wrapping both in a Hugging Face pipeline gives a simple callable interface — you just call `self.model(text)` and get a result back.

Why text is split into chunks

Language models have a maximum input length. Rather than truncating long transcripts, `_split_into_chunks()` breaks the transcript into pieces of roughly 80 words each, splitting at sentence boundaries (periods) so no sentence is cut in half. Each chunk is simplified independently, then all simplified chunks are joined back into one final text.

Prompt engineering

FLAN-T5 is an instruction-following model — it responds to natural-language instructions rather than requiring special formatting. The prompt sent for every chunk is:

```
prompt = (
    "Rewrite this text in very simple English that is easy to "
    "understand. Use short sentences. Use common everyday words. "
    "Keep the same meaning.\n\n"
    f"Text: {text}"
)
```

This single sentence of instructions is what steers the model toward short sentences and everyday vocabulary. If your team trains a custom model on a specific instruction format, this is the exact place to update it to match.

Generation settings explained

Setting	Value	Effect
max_length	130	Maximum number of tokens the model is allowed to generate per chunk.
min_length	30	Minimum output length — prevents overly short, low-effort outputs.
num_beams	4	Beam search width. Higher values explore more possible outputs and usually improve quality, at the cost of speed.
do_sample	False	Deterministic generation — the same input always produces the same output, which is important for reproducible results.

4.4 agents/summarization_agent.py — Agent 3

Purpose: condenses the transcript down to its key points. Unlike simplification (same length, simpler words), summarization produces a *shorter* text.

Libraries used: transformers, using the "summarization" pipeline task (as opposed to "text2text-generation" used by the simplification agent). The default model is sshleifer/distilbart-cnn-12-6, a distilled (smaller, faster) version of Facebook's BART model, pretrained specifically for summarizing news-style text.

Chunking strategy — why it differs from Agent 2

The chunk size here is 500 words by default, compared to 80 words for simplification. This reflects the underlying models: FLAN-T5 base and DistilBART have different maximum input lengths, and summarization models are generally trained to compress much longer input spans, whereas the simplification prompt works best on shorter, sentence-focused chunks.

```
def _split_into_chunks(self, text):
    chunk_size = self.config["chunk_size"] # 500 words, from config.py
    words = text.split()
    return [
        " ".join(words[i:i + chunk_size])
        for i in range(0, len(words), chunk_size)
    ] or [text]
```

Skipping short chunks

If a chunk has fewer than 30 words, it is too short to meaningfully summarize (the model would just repeat it back or produce nonsense). The agent detects this and keeps the chunk as-is instead of running it through the model:

```
if len(chunk.split()) < 30:
    summary_parts.append(chunk) # too short – skip summarization
    continue

result = self.model(
    chunk,
    max_length=self.config["max_length"],
    min_length=self.config["min_length"],
    do_sample=False,
)
summarized = result[0]["summary_text"]
```

The compression_ratio metric

Every summarization result includes a `compression_ratio` — the summary's word count divided by the original transcript's word count. A ratio of 0.25 means the summary is a quarter of the length of the original, which is a useful, concrete number to show in a project discussion when demonstrating how much the pipeline condenses content.

```
"compression_ratio": round(  
    len(summary.split()) / max(len(transcript.split()), 1), 3  
)
```

The `max(..., 1)` guards against a division-by-zero error if the transcript is somehow empty.

4.5 core/orchestrator.py

Purpose: the coordinator. Loads all three agents once, then runs them in sequence for every audio file it's given, collecting and combining their results.

Initialization — loading all agents once

Model loading is slow (each model can take 10–30 seconds to load into memory). The `VoxaOrchestrator` class loads all three agents exactly once, in its constructor, not on every call to `.run()`:

```
def __init__(self):
    self.transcription_agent = TranscriptionAgent()
    self.simplification_agent = SimplificationAgent()
    self.summarization_agent = SummarizationAgent()
```

This means you create **one** orchestrator object at the start of your program, and can then call `.run()` on many different audio files without reloading the models each time.

The `run()` method — step by step

- **Stage 1:** Calls `transcription_agent.transcribe(audio_path)`. If the resulting transcript is empty, the pipeline stops early and returns an error result — there's no point simplifying or summarizing nothing.
- **Stage 2:** Calls `simplification_agent.simplify(transcript)`.
- **Stage 3:** Calls `summarization_agent.summarize(transcript)`.
- **Consolidation:** merges all three results, plus metadata (confidence, word counts, compression ratio, elapsed time) into one dictionary.
- **Persistence:** saves that dictionary as a JSON file into `data/output/`, named after the input audio file.

Error handling

The entire pipeline is wrapped in a single `try/except` block. If any agent raises an exception — a corrupted audio file, a missing model, an out-of-memory error — the Orchestrator catches it and returns a clearly structured error result instead of crashing the whole program:

```
def _error_result(self, error_message: str) -> dict:
    print(f"\n[Orchestrator] ✗ ERROR: {error_message}")
    return {
        "success": False,
        "error": error_message,
        "transcript": "",
        "simplified_text": "",
        "summary": "",
    }
```

The caller (whether that's `run_pipeline.py` or the Streamlit app) always checks `result["success"]` before trying to display any text, which is what prevents the UI from showing a blank or broken result when something goes wrong.

4.6 run_pipeline.py

Purpose: a command-line alternative to the web interface. Useful for quick testing, automation, or running the pipeline on a server without a browser.

Library used: argparse, Python's standard library for parsing command-line arguments.

```
parser.add_argument("audio_path", type=str,
                    help="Path to the audio file (.wav, .mp3, .flac, etc.)")
parser.add_argument("--output", type=str, default=None,
                    help="Optional custom path to save the JSON result")
```

Usage:

```
python run_pipeline.py data/input/my_audio.wav
python run_pipeline.py data/input/my_audio.wav --output results.json
```

The script builds one `VoxaOrchestrator`, calls `.run(audio_path)` once, and prints the transcript, simplified text, and summary directly to the terminal — the exact same result object the Streamlit UI uses, just displayed differently.

4.7 ui/streamlit_app.py

Purpose: the web interface. Lets a user upload an audio file, click a button, and see all three outputs in the browser, with download options.

Library used: streamlit, a Python framework that turns a script into a web app without writing any HTML, CSS, or JavaScript.

Caching the orchestrator

Streamlit re-runs the entire script from top to bottom every time the user interacts with the page (uploads a file, clicks a button, etc.). Without special handling, this would reload all three AI models on every click — unacceptably slow. The `@st.cache_resource` decorator solves this by keeping the loaded orchestrator in memory across reruns:

```
@st.cache_resource(show_spinner=False)
def load_orchestrator():
    return VoxaOrchestrator()
```

The models are loaded exactly once, the first time a user processes an audio file, and reused for every subsequent upload during that session.

The upload → process → display flow

- `st.file_uploader()` renders a drag-and-drop box restricted to audio file types (.wav, .mp3, .flac, .ogg, .m4a).
- The uploaded file is written to a temporary file on disk using Python's `tempfile` module, since the agents expect a file path, not raw bytes.
- Clicking “Process Audio” calls `orchestrator.run(temp_audio_path)`.

- The result dictionary is stored in `st.session_state`, Streamlit's mechanism for remembering data across reruns — this is why the results stay visible even after the page refreshes from other interactions.
- Three styled cards display the transcript, simplified text, and summary, each with relevant metadata badges (confidence, word counts, compression ratio).
- Three download buttons let the user save the transcript and simplified text as .txt files, and the full result as a .json file.

```
with tempfile.NamedTemporaryFile(
    delete=False,
    suffix=os.path.splitext(uploaded_file.name)[1]
) as tmp_file:
    tmp_file.write(uploaded_file.getbuffer())
    temp_audio_path = tmp_file.name

orchestrator = load_orchestrator()
result = orchestrator.run(temp_audio_path)
os.remove(temp_audio_path)  # clean up the temp file afterwards
```

Styling approach

The interface uses `st.markdown()` with `unsafe_allow_html=True` to inject custom CSS — a light background, rounded card containers, a restrained neutral color palette, and a dark primary button. This keeps the interface calm and readable, which matters directly for an accessibility-focused tool: high contrast text, generous spacing, and no visual clutter.

5. Libraries & Technologies Used

This section explains every major library the project depends on, what it does, and why it was chosen.

5.1 Core AI Models

Library / Model	Role in Voxa
openai-whisper	OpenAI's automatic speech recognition (ASR) model. Converts raw audio into text. Chosen for strong accuracy across many accents and languages, and because it runs entirely offline once downloaded.
torch (PyTorch)	The deep learning framework underneath both Whisper and the Hugging Face models. Handles all the numerical computation (tensor operations, neural network layers) on CPU or GPU.
transformers	Hugging Face's library for loading and running pretrained language models. Provides the FLAN-T5 and DistilBART models, plus the tokenizer and pipeline utilities.
FLAN-T5 (google/flan-t5-base)	An instruction-tuned text-to-text model used for simplification. It was fine-tuned by Google to follow natural-language instructions, which is why a plain-English prompt is enough to steer its output.
DistilBART (sshleifer/distilbart-cnn-12-6)	A distilled (smaller, faster) version of Facebook's BART model, pretrained specifically for abstractive summarization of news-style text.

5.2 Audio Processing

Library	Role in Voxa
librosa	Loads audio files in almost any format and resamples them to the 16 kHz rate Whisper requires, regardless of the original recording's sample rate.
soundfile	A lower-level audio I/O library that librosa relies on internally for reading and writing audio data.
ffmpeg (system tool)	A command-line program (not a Python package) that Whisper calls internally to decode compressed audio formats like MP3 and M4A. Must be installed separately on the operating system.

5.3 Text Processing & Utilities

Library	Role in Voxa
sentencepiece	The tokenizer format used by T5-based models. Required for FLAN-T5 to convert text into tokens.
accelerate	A Hugging Face utility that optimizes how models are loaded and run across CPU/GPU, used internally by transformers.
nltk	Natural Language Toolkit. Used for sentence-boundary detection when chunking long transcripts.
jiwer	Computes Word Error Rate (WER), a standard metric for evaluating transcription accuracy against a reference transcript.
pandas	Used for structuring and saving batch results as CSV files when processing multiple audio files.
json (standard library)	Used throughout the project to save structured results to disk in a human-readable, language-agnostic format.

5.4 Web Interface

Library	Role in Voxa
streamlit	Turns the Python script in ui/streamlit_app.py into a fully interactive web application — file upload, buttons, styled result cards, and download buttons — without writing any HTML, CSS, or JavaScript directly.

6. Setup & Running the Project

6.1 Environment Setup

- Install Python 3.10 or 3.11.
- Create a virtual environment: `python -m venv venv`, then activate it.
- Install ffmpeg as a system program (not pip) — required by Whisper to decode audio.
- Install all Python dependencies: `pip install -r requirements.txt`.
- Download NLTK sentence-tokenizer data: `python -c "import nltk; nltk.download('punkt')"`.

6.2 Adding Trained Models

By default, `use_pretrained: True` in `config.py`, so the pipeline runs immediately using public models — useful for testing before your team's trained models are ready. To use your own trained model:

- Save the model with `model.save_pretrained("models/simplification_model")` (or the equivalent folder for summarization).
- Confirm the folder contains all required files: `config.json`, `model.safetensors`, `tokenizer.json`, `tokenizer_config.json`, `special_tokens_map.json`, and (for T5-based models) `spiece.model`.
- Set `use_pretrained: False` for that agent in `config.py`.

6.3 Running the Web Interface

```
streamlit run ui/streamlit_app.py
```

Opens a browser tab at `http://localhost:8501`. Upload an audio file, click “Process Audio”, and view the transcript, simplified text, and summary.

6.4 Running from the Command Line

```
python run_pipeline.py data/input/my_audio.wav
```

6.5 Testing Individual Agents

Each agent file can be run directly to test it in isolation, without loading the other two agents:

```
python agents/transcription_agent.py
python agents/simplification_agent.py
python agents/summarization_agent.py
python core/orchestrator.py
```

7. Team Roles & Responsibilities

The 3-agent architecture maps directly onto a 5-person team, giving each member a clearly scoped, independently testable piece of the system.

Role	Owns	Key Deliverable
Person 1	agents/transcription_agent.py	Working audio-to-text agent, tested standalone with a sample .wav file.
Person 2	agents/simplification_agent.py	Working text-simplification agent, tested with sample transcript text.
Person 3	agents/summarization_agent.py	Working summarization agent, tested with sample transcript text.
Person 4	core/orchestrator.py, run_pipeline.py, config.py	Full pipeline integration and command-line tool.
Person 5	ui/streamlit_app.py, README.md	Web interface and full project documentation.

7.1 Integration Contract

For the 3 agents to plug into the Orchestrator without modification, each one follows an agreed-upon interface:

- `TranscriptionAgent.transcribe(audio_path)` must return a dict containing a "transcript" key.
- `SimplificationAgent.simplify(transcript)` must return a dict containing a "simplified_text" key.
- `SummarizationAgent.summarize(transcript)` must return a dict containing a "summary" key.

As long as each agent honors this contract, its internal implementation — which model it loads, how it chunks text, what prompt it uses — can be changed freely without affecting the Orchestrator or the UI.

8. Future Improvements

Possible directions to extend the project beyond the current scope:

- **Speaker diarization:** use a library such as `pyannote.audio` to identify “Speaker 1 said...” vs “Speaker 2 said...”, useful for meetings and conversations with multiple people.
- **Subtitle export:** generate a standard `.srt` subtitle file from Whisper's timed segments, so results can be loaded directly into video players like VLC.
- **Real-time transcription:** add a microphone-input agent using `sounddevice` or `pyaudio`, so Voxa can process live audio instead of only pre-recorded files.
- **Larger Whisper model:** switching from Whisper-base to Whisper-small or Whisper-medium in `config.py` measurably improves transcription accuracy at the cost of slower processing.
- **Confidence-based filtering:** automatically flag or exclude low-confidence transcript segments before they reach the simplification and summarization stages.
- **Batch processing mode:** extend `run_pipeline.py` to accept a folder of audio files and process all of them in one run, logging results to a single CSV.